

FITBUDDY

AI Fitness Plan Generator using Gemini Models

REQUIREMENT ANALYSIS PHASE

This document presents the requirement analysis for the FitBuddy project. It identifies the users, inputs, functional requirements, AI requirements, database requirements, interface requirements, technical requirements and expected system behavior based on the project documentation.

Requirement Area	Identified Direction
Application	Web-based AI fitness planning application
Main Output	Personalized 7-day workout plan
Supporting Output	Nutrition or recovery tip
Adaptive Feature	Feedback-based workout plan update
Storage	SQLite with SQLAlchemy
Backend	FastAPI
Frontend	HTML + Jinja2
AI	Google Gemini models

Analysis Objective: Convert the FitBuddy concept into clear and implementable requirements that guide system design, development, testing and deployment.

1. Requirement Analysis Overview

Requirement analysis defines what the FitBuddy application is expected to do and what information it needs to perform its functions. The project documentation identifies personalized workout generation, nutrition guidance, feedback-based updating, data storage and administrative viewing as the major areas.

1.1 Main Objectives

- Collect user fitness information through a web form.
- Generate a structured 7-day workout plan based on the user's goal and intensity.
- Provide a concise nutrition or recovery tip.
- Allow the user to submit feedback about the existing plan.
- Generate an updated plan using the original plan and feedback.
- Store user information and generated plans.
- Allow administrators or coaches to view users and plans.

1.2 Scope of the Application

The documented scope covers the user input interface, AI-powered generation, feedback-based revision, database storage, result display and an administrative view. The documentation does not specify additional features beyond these areas, so they are not included as required functions here.

1.3 Requirement Sources

Requirements are derived from the project description, four documented usage scenarios, technical architecture, core functionality descriptions, route descriptions, frontend descriptions and deployment requirements.

2. User and Stakeholder Requirements

The project documentation identifies two broad types of users: normal users who receive and refine fitness plans, and administrators or coaches who can view stored users and plans.

User / Stakeholder	Requirement
Fitness User	Enter personal information and fitness preferences.
Fitness User	Receive a personalized 7-day workout plan.
Fitness User	Receive a nutrition or recovery tip.
Fitness User	Submit feedback to revise the plan.
Administrator / Coach	View registered users and their plans.
Administrator / Coach	View original and updated plans.

2.1 User Input Requirements

- Username / name
- User ID
- Age
- Weight
- Fitness goal such as weight loss or muscle gain
- Workout intensity: low, medium or high

2.2 Feedback Requirements

The feedback function must accept a user ID and feedback about the existing workout plan. Examples documented include requests for more cardio, more yoga or additional rest days.

3. Functional Requirements

Functional requirements describe the actions the system must perform. They are based directly on the functions and routes described in the project documentation.

ID	Functional Requirement	Description
FR-01	User Input	System shall accept user information through the HTML form.
FR-02	Workout Generation	System shall generate a personalized 7-day workout plan.
FR-03	Nutrition Tip	System shall generate a concise nutrition/recovery tip.
FR-04	Feedback	System shall accept feedback for an existing plan.
FR-05	Plan Update	System shall generate a revised plan from the original plan and feedback.
FR-06	Save User	System shall store user details in the database.
FR-07	Save Plan	System shall store generated plans.
FR-08	Admin View	System shall display registered users and their plans.

3.1 Core Functions

- `generate_workout_gemini()` – generates the personalized workout plan.
- `generate_nutrition_tip_with_flash()` – generates the nutrition or recovery tip.
- `update_workout_plan()` – revises the original plan using feedback.
- `save_user()`, `save_plan()`, `update_plan()` – manage stored user and plan information.
- `get_original_plan()`, `get_user()` – retrieve stored information.

4. Input, Processing and Output Requirements

The system can be analyzed through three stages: user input, AI/database processing and result output.

Stage	Requirements
Input	Name, user ID, age, weight, goal and intensity.
Processing	Validate form data, construct user input, call appropriate Gemini functions and save records.
Workout Output	Structured 7-day plan with warm-up, main workout and cooldown/recovery guidance.
Nutrition Output	Concise practical nutrition or recovery suggestion.
Feedback Input	User ID and feedback describing desired changes.
Updated Output	Revised workout plan and updated nutrition tip.

4.1 Workout Plan Requirements

- Plan must be organized day by day.
- Exercises should include details such as sets, repetitions or duration.
- The plan may include warm-up and cooldown/recovery guidance.
- The plan should reflect the selected fitness goal and intensity.

4.2 Nutrition / Recovery Requirements

The nutrition function should return a concise, practical suggestion related to the selected fitness goal. The documented examples include hydration, protein suggestions and general recovery practices.

5. Feedback and Plan Update Requirements

Feedback-based updating is a central adaptive requirement of FitBuddy. The user is not limited to the initially generated plan and can request changes.

5.1 Feedback Process

- User provides their User ID.
- User enters feedback about the existing plan.
- The system retrieves the original plan from the database.
- The original plan and feedback are sent to the update function.
- Gemini generates a revised plan.
- The updated plan is stored separately from the original plan.
- The updated result is displayed to the user.

5.2 Examples of Feedback

- Add more cardio.
- Include yoga.
- Include more rest days.
- Change the difficulty of the workout.
- Adjust the plan according to user preferences.

5.3 Data Preservation Requirement

The documentation specifies that feedback-based updates are saved as a separate field so that both the original and updated versions can be preserved.

6. AI and Technology Requirements

The requirement analysis identifies the technologies needed to implement the documented functions.

Component	Technology	Requirement
AI Workout / Update	Gemini 1.5 Pro	Generate structured workout plans and revise plans.
AI Nutrition	Gemini Flash	Generate fast nutrition or recovery tips.
Backend	FastAPI	Manage routing, form processing and application logic.
Frontend	HTML + Jinja2	Collect user input and display dynamic results.
Database	SQLite + SQLAlchemy	Store users and original/updated plans.
Server	Uvicorn	Run the FastAPI application.

6.1 Environment Requirements

- Python and pip must be available.
- A virtual environment should be used.
- Required libraries include FastAPI, Uvicorn, Jinja2, SQLAlchemy, python-multipart and Google Generative AI.
- The Gemini API key must be configured through an environment variable or .env file.
- The application should be runnable through Uvicorn.

6.2 API Requirements

The application requires Gemini API integration for workout generation, nutrition-tip generation and feedback-based plan updates. FastAPI also provides interactive API documentation at the documented /docs endpoint.

7. Database and Data Requirements

The documentation specifies SQLite with SQLAlchemy ORM for persistence. The database requirement is to retain user information and AI-generated plans.

7.1 User Data

Data	Purpose
User ID	Identify the user.
Name / Username	Display and identify the user.
Age	Part of the user's fitness information.
Weight	Part of the user's fitness information.
Goal	Determine the fitness objective.
Intensity	Determine preferred workout intensity.

7.2 Plan Data

Plan Data	Requirement
Original Plan	Store the initial AI-generated workout plan.
Updated Plan	Store the revised plan after feedback.
Nutrition Tip	Store/display the supporting tip as part of the result.
Feedback	Use feedback as input for plan revision.

7.3 Administrative Data Requirement

The /view-all-users route should retrieve users and plans so that the administrative page can display username, age, weight, goal, intensity and original/updated plans.

8. Interface and Route Requirements

The frontend requirements describe three main templates and the FastAPI routes that connect them.

Route	Purpose	Template / Output
/	Display the home form.	index.html
/generate-workout	Process user input and generate workout + nutrition tip.	result.html
/submit-feedback	Process feedback and update the plan.	result.html
/view-all-users	Display all users and their plans.	all_users.html

8.1 Interface Requirements

- index.html must collect the documented user fields.
- result.html must display user information, workout plan and nutrition tip.
- result.html must provide a feedback form.
- all_users.html must display stored users and plans.
- Jinja2 should be used to bind backend data to the HTML templates.
- The interface should be responsive and provide readable result blocks.

8.2 Backend Processing Requirements

FastAPI routes must receive and process form data, construct the documented Pydantic input models, call the appropriate AI functions, save or update database records, and render the required template.

9. Non-Functional, Deployment and Acceptance Requirements

The available project documentation provides several quality and deployment expectations. These are included here without adding requirements that are not documented.

9.1 Interface and Quality Expectations

- The interface should be clean and user-friendly.
- The frontend should support responsive layouts.
- Forms and result blocks should be clearly structured.
- AI-generated results should be displayed in an understandable format.
- The system should support real-time web-based interaction as a model requirement.

9.2 Deployment Requirements

- Create and activate a Python virtual environment.
- Install dependencies from requirements.txt.
- Configure the Gemini API key using an environment variable or .env file.
- Start the FastAPI application using Uvicorn.
- Verify the application through the local browser.
- Use the documented /docs interface for API testing.

9.3 Requirement Acceptance Checklist

Check	Expected Result
User form	Required user and fitness information can be entered.
Workout generation	A personalized 7-day plan is produced.
Nutrition tip	A relevant concise tip is produced.
Feedback	User can submit plan feedback.
Plan update	A revised plan is generated from original plan + feedback.
Database	User and plan records are stored.
Admin page	Users and original/updated plans can be viewed.
Local deployment	Application runs through the documented Uvicorn setup.

10. Conclusion

The requirement analysis establishes the functional, data, interface, AI, technology and deployment requirements for FitBuddy. It provides the foundation for the next project phases, including system design, implementation, testing and deployment.

END OF REQUIREMENT ANALYSIS PHASE